Linear Layer Mathematical Verification

A rigorous breakdown of the calculus and indexing transformations within the custom C++ Fully Connected Layer.

1. Structural Architecture & Variable Mapping

In deep learning frameworks, matrices are flattened into continuous 1D arrays to ensure memory locality and cache efficiency. This implementation structures its multi-dimensional spaces using the following explicit dimensional paradigms:

X (inp): The input matrix representing features mapped across a continuous execution stream. Dimension: **(Batch Size × N_in)**.

W (weights): The parameter connection topology mapping relationships from input fields to target outputs. Dimension: **(N_in × N_out)**.

b (bias): The stationary translation offset matrix mapping directly to out-stream channels. Dimension: **(1 × N_out)**.

Y (out): The final computed activation grid, or the incoming localized backpropagation error tracking matrix. Dimension: **(Batch Size × N_out)**.

2. Linear Forward Pass Transformations

The forward step combines multi-dimensional weight matrices and bias shifts into a centralized matrix operation. The global algebraic equation maps as:

$$Y = X \times W + b$$

For any individual matrix location isolated at row coordinate *i* and output neuron position *k*, the evaluation operates on scalar elements via:

$$Y(i, k) = b(k) + \sum_{[j = 0 \text{ to } N_in - 1]} [X(i, j) \times W(j, k)]$$

To reconcile this formula against flattened 1D continuous row-major memory partitions, the tracking pointer indexes are evaluated dynamically inside the loop structure:

$$ind_out = i \times N_out + k$$
$$ind_inp = i \times N_in + j$$
$$ind_weights = j \times N_out + k$$

3. Multi-Variable Calculus Backpropagation Steps

During the localized backward sweep, the execution block intercepts the error gradient with respect to outputs (represented by the **out** variable pointer) and applies the multivariate chain rule to yield the loss gradient with respect to inputs:

$$\partial L / \partial X(i, j) = \sum_{[k = 0 \text{ to } N_out - 1]} [(\partial L / \partial Y(i, k)) \times (\partial Y(i, k) / \partial X(i, j))]$$

Differentiating the core linear forward transform establishes that ∂Y(i, k) / ∂X(i, j) simplifies exclusively to the structural connection coefficient W(j, k). Substituting this element simplifies the matrix transformation formula to:

$$\partial L / \partial X(i, j) = \sum_{[k = 0 \text{ to } N_out - 1]} [(\partial L / \partial Y(i, k)) \times W(j, k)]$$

Expressed in global matrix notation, this maps perfectly to a transposed matrix dot-product routine executed across VRAM zones:

$$\partial L / \partial X = (\partial L / \partial Y) \times W^T$$

Because the tracking space is cleared via an initialization block prior to calculation execution, accumulating values sequentially inside the inner loops cleanly establishes the exact total derivative matrix.

4. Parameter Gradient Minimization Steps

To minimize the objective loss function across continuous training iterations, parameters are modified in place along the steepest descent vectors according to a designated learning rate modifier (η):

Bias Updates: Since bias coefficients act as linear transformations with static scaling shifts (∂Y(i, k) / ∂b(k) = 1), the accumulated total gradient reflects basic summation tracks evaluated across individual sample dimensions:

$$\partial L / \partial b(k) = \sum_{[i = 0 \text{ to } Batch_Size - 1]} [\partial L / \partial Y(i, k)]$$

$$b(k) \leftarrow b(k) - \eta \times (\partial L / \partial b(k))$$

Weight Matrix Updates: Applying the chain rule defines the optimization derivative matrix mapping back to the dot product of input matrices and output errors:

$$\partial L / \partial W(j, k) = \sum_{[i = 0 \text{ to } Batch_Size - 1]} [X(i, j) \times (\partial L / \partial Y(i, k))]$$

$$W(j, k) \leftarrow W(j, k) - \eta \times (\partial L / \partial W(j, k))$$

In global matrix terms, this equates cleanly to:

$$\partial L / \partial W = X^T \times (\partial L / \partial Y)$$

5. Structural Safety Architecture: The cp_weights Paradigm

In your class structure, a duplicate shadow allocation segment named **cp_weights** is explicitly maintained during parameter adjustment execution. This is a vital mathematical constraint.

Because parameter weight adjustments happen in place inside the loop systems, executing an optimization update directly modifies the weight values before backpropagation processing completes. If you did not buffer the old configuration values within a protected array state, the backward pass calculations would evaluate their error propagation gradients using corrupted, post-update weight figures. This breaking change breaks mathematical consistency, destabilizes the chain rule progression, and destroys overall network convergence properties.